

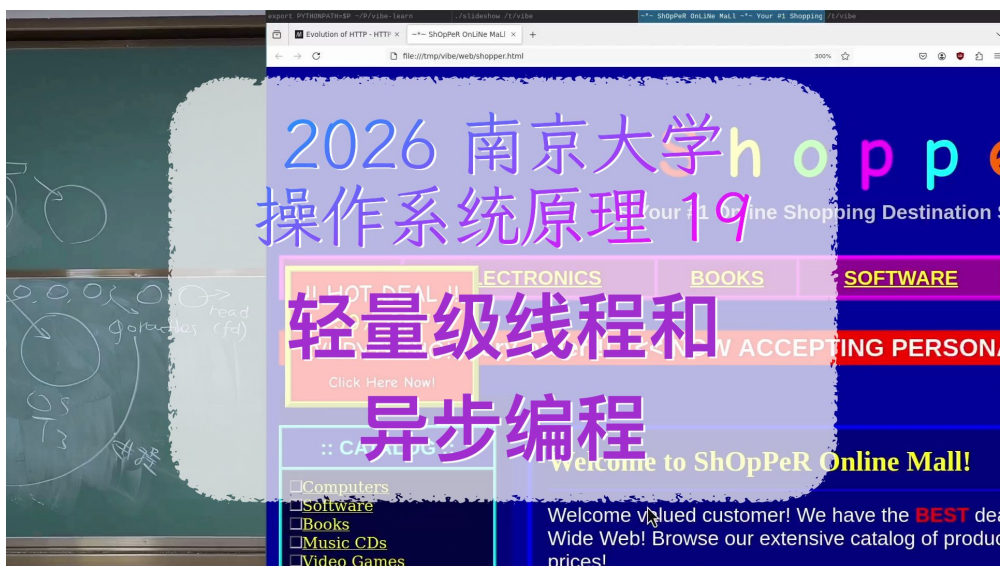
课程笔记

协程、Goroutine、异步编程

2026 南京大学《操作系统原理》第 19 讲

lecture-to-notes

2026 年 5 月 18 日



视频作者/频道：绿导师原谅你了（蒋炎岩 / jyy）

发布日期：2026-05-09

视频时长：1:22:36

视频链接：<https://www.bilibili.com/video/BV1dVRdBpEze/>

目录

1 本讲定位：让”线程”变得 真的便宜	2
1.1 为什么 OS 线程太贵?	2
1.2 两条路：lightweight thread vs async	3
1.3 本章小结	3
2 路径一：协程 / Goroutine ——让线程变轻	4
2.1 Python Generator：你已经用过的协程	4
2.1.1 在 C 里怎么实现? setjmp / longjmp	4
2.2 Go 的 goroutine：M:N 调度	5
2.3 协程的陷阱：当用户态调度遇到内核态阻塞	5
2.3.1 非阻塞 I/O 的解决方法	6
2.4 本章小结	6
3 路径二：JavaScript / Promise / async-await	7
3.1 为什么浏览器选了”完全单线程”?	7
3.2 Callback Hell → Promise → async/await	8
3.3 从计算图视角统一所有这些机制	9
3.4 本章小结	9
4 总结与延伸	10
4.1 讲者核心观点	10
4.2 自测三问	10
4.3 与课程其他部分的 synthesis	10
4.4 拓展阅读	10

1 本讲定位：让”线程”变得 真的便宜

上一讲（第 18 讲）告诉我们：”并行算法 = 计算图设计”——把同步压在关键路径上、本地计算尽可能多。但有一个隐含假设：**创建一个线程的代价 $\ll$ 它能承担的计算量**。一旦我们想画的计算图有几百万个微小节点，OS 线程（每个 8 MB 栈 + 内核调度）就贵到根本用不起。

本讲解决的问题

怎样让”启动一个并行单元的代价 $\approx$ 一次函数调用”？

讲者给出**两条路**：

1. 把”线程”做轻——用户态调度的**协程** (coroutine) / Goroutine，保持多线程编程模型不变；
2. 把”程序的执行模型”改掉——用 **Promise / async-await** 直接在代码里描述计算图。

两条路殊途同归——都是要把 100 万个”并发执行单元”塞进一个进程里。

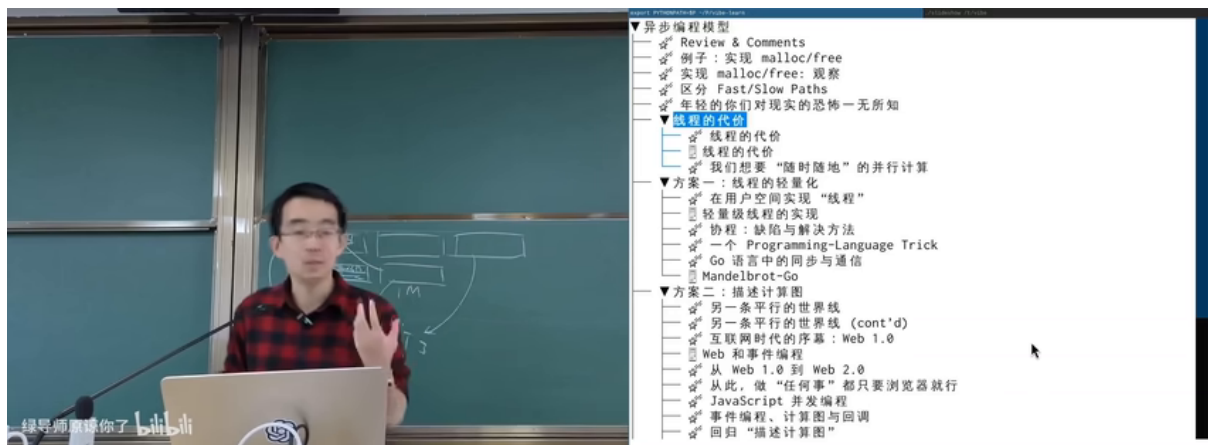


图 1：本讲的完整 outline：从上一讲的 malloc/free 收尾（fast/slow path），切入”线程的代价”问题；两条方案（轻量化线程 = 协程/goroutine；异步编程 = Promise/async/await）展开；最后用 JavaScript / Web 1.0→2.0 演化作为”在浏览器里见过这个事情的人”的具体落地¹

1.1 为什么 OS 线程太贵？

回忆第 13 讲讲到的：`pthread_create` 出来的线程默认占 **8 MB 虚拟地址空间**（栈），还要在内核里登记一个 `task_struct`，挂到调度队列上。具体代价：

- **时间**：单次 `create` + `join` 微秒级；
- **空间**：每个 `1e4` 线程吃掉 80 GB 虚拟地址；
- **上下文切换**：内核态切换有 TLB miss / cache cold 等隐藏开销。

如果你要做一个”画一张 Mandelbrot 图，每个像素一个并行任务”——那是百万级任务。**用 OS 线程根本不可能**——你必须自己把任务批量化打包。这个”批量化打包”的活，能不能让编译器 / 运行时帮你？这就是本讲的全部。

¹ 视频画面时间区间：00:13:30–00:13:45。

1.2 两条路：lightweight thread vs async

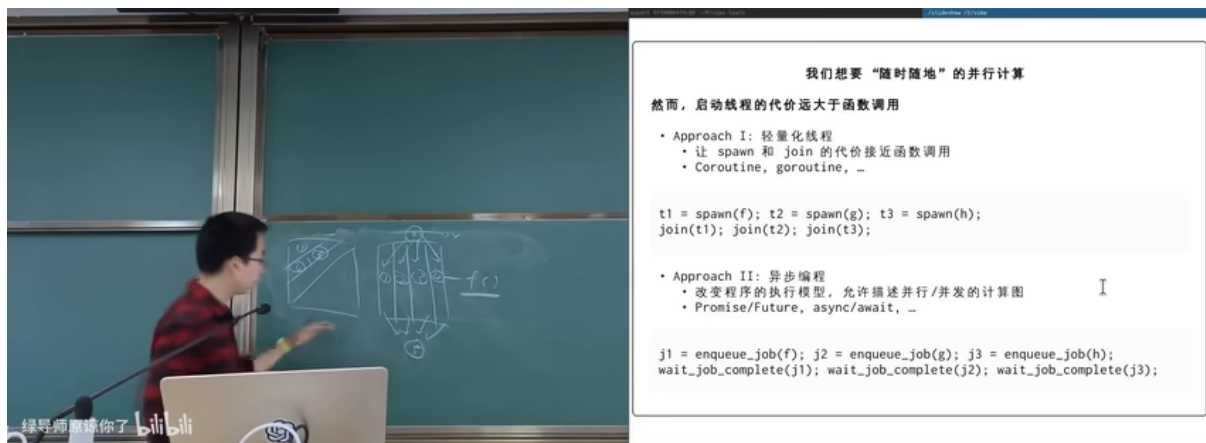


图 2: ”随时随地并行计算”的两条工程路径。**Approach I (轻量化线程)**: 保持多线程语法不变, 让 spawn/join 的代价接近函数调用——**Coroutine, goroutine, ...**

Approach II (异步编程): 改变程序的执行模型, 允许代码直接描述并行/并发的计算图——**Promise/Future, async/await, ...**²

写法对比:

```
1 // Approach I: 轻量化线程
2 t1 = spawn(f); t2 = spawn(g); t3 = spawn(h);
3 join(t1); join(t2); join(t3);
4
5 // Approach II: 异步任务队列
6 j1 = enqueue_job(f); j2 = enqueue_job(g); j3 = enqueue_job(h);
7 wait_job_complete(j1); wait_job_complete(j2); wait_job_complete(j3);
```

Listing 1: 两种 API 的形态

API 几乎一样——区别在于运行时 / 语言怎么实现它们: 轻量化线程靠用户态栈切换 (不进内核), 异步编程靠编译器把代码改写成状态机。

1.3 本章小结

本讲不引入复杂同步原语, 而是引入两种”重新设计并发单元”的思路。”线程”在本讲会被替换成 **协程** (用户态调度) 或 **异步任务** (事件循环驱动), 并发量瞬间从”几千”提升到”几百万”。

²视频画面时间区间: 00:23:45–00:24:00。

2 路径一：协程 / Goroutine ——让线程变轻

2.1 Python Generator：你已经用过的协程

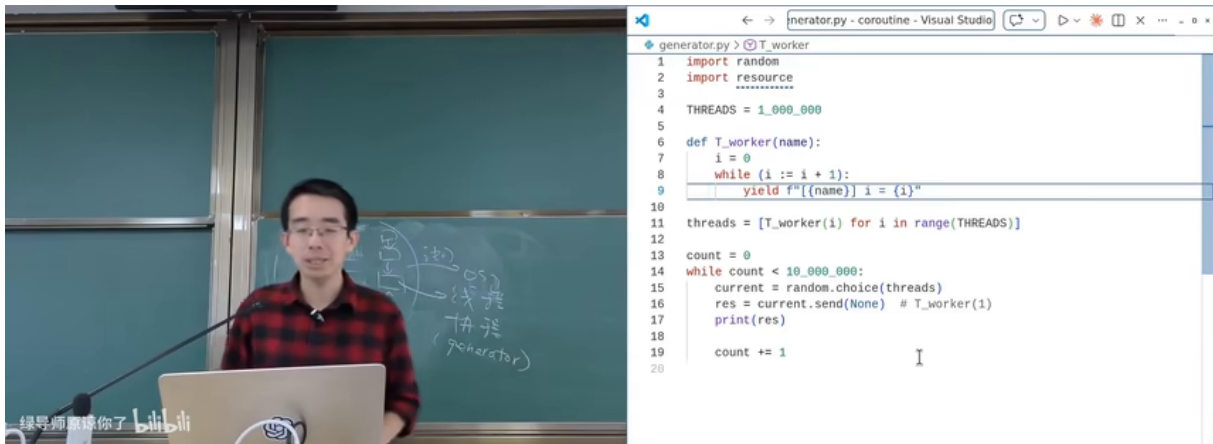


图 3: Python generator 演示在单线程里跑 100 万个”协程”：T_worker 是一个无限循环 + yield；[T_worker(i) for i in range(1_000_000)] 创建 100 万个 generator 对象（此时没有任何线程被启动）；主循环用 random.choice 随机挑一个 current.send(None) 跑一步——跑到 yield 就把当前协程的执行状态冻结，回到主循环³

Generator

回到 simple C：单线程程序 = ”一个栈 + 一个 PC”；多线程程序 = ”多个栈 + 每栈一个 PC”。Python generator 给了你一个进程里 100 万个栈，全部由你手动调度——关键就在 yield：它冻结当前栈的 PC，把控制权交回调用者。下一次 send() 时再从冻结点恢复执行。这就是用户态线程的本质。

2.1.1 在 C 里怎么实现？setjmp / longjmp

```
1 #include <setjmp.h>
2 jmp_buf coro_env;
3
4 void coroutine() {
5     if (setjmp(coro_env) == 0) {
6         // 第一次进入：保存上下文，准备运行
7         do_something();
8         longjmp(main_env, 1); // yield 回 main
9     }
10    // longjmp 回来后从这里继续...
11 }
```

Listing 2: C 的非局部跳转 + 协程骨架

setjmp 把当前所有调用者保存的寄存器 (SP、PC、callee-saved regs) 写到 jmp_buf 里；longjmp 把它们恢复回来。”切换协程”≡”swap 两个 jmp_buf”。

³视频画面时间区间：00:31:00-00:31:15。

Protothreads / Boost.Context

更工业级的实现叫 **Protothreads** (用 `switch-case` 状态机模拟) 和 **Boost.Context** (直接用汇编 `swap` 寄存器)。Linux 上还有 `makecontext` / `swapcontext`——这一族 API 都是同一种思路：在用户态把”切换执行流”做成几条汇编指令，远比 `pthread_*` 的进内核要便宜。

2.2 Go 的 goroutine: M:N 调度

Go 把”轻量化协程”做成了语言级别的特性：

- `go func() { ... }()` 启动一个 goroutine，开销 ~ 200 ns；
- 初始栈只有 **2 KB** (按需自动增长 / 收缩)；
- 单进程几百万个 goroutine 是常态；
- 运行时调度器维护 **M:N 模型**——M 个 OS 线程 (GOMAXPROCS，默认 = CPU 核数) 调度 N 个 goroutine。

M:N 调度的精髓

- **M (machine)**: 真正吃 CPU 的内核线程。
- **P (processor)**: 逻辑调度单元 (GOMAXPROCS 决定数量)。
- **G (goroutine)**: 用户态任务。

M 在每个调度点 (`channel` 操作、阻塞 `syscall`、函数调用栈检查点) 主动让出 P 给下一个 G。所以 goroutine 是协作式 + 部分抢占的——不像 OS 线程那样靠时钟中断强制抢占。

2.3 协程的陷阱：当用户态调度遇到内核态阻塞

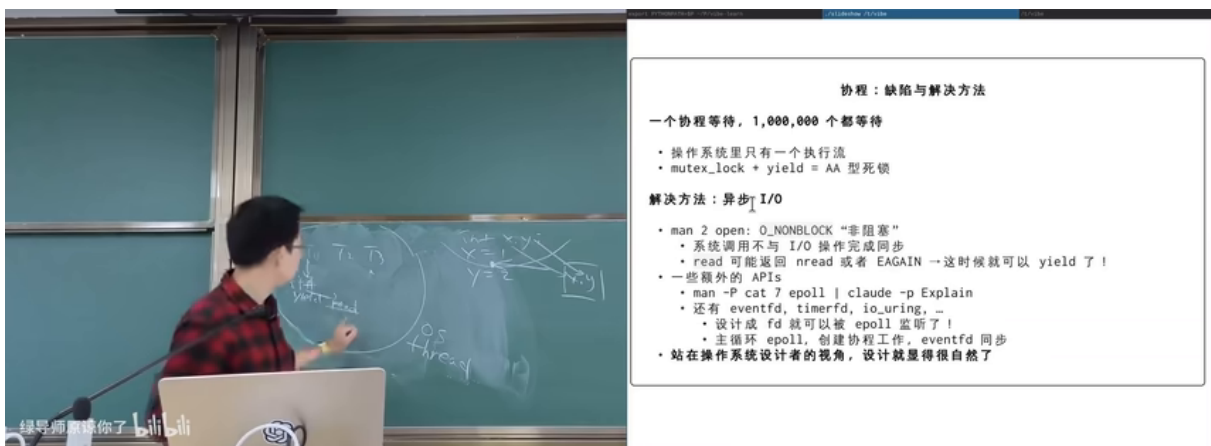


图 4: 协程的两大缺陷：(1) 一个协程阻塞 $\rightarrow$ 全部死等——操作系统只看到一个执行流，`read()` 没有数据时整个进程都停下；(2) 同 `thread` 内 `mutex_lock + yield = AA` 型死锁——锁还在自己手里，但调度器把控制权交给同一个协程，结果它再次 `lock` 就死。解决方案：异步 I/O——所有阻塞系统调用换成 `O_NONBLOCK + epoll / io_uring / eventfd`⁴

2.3.1 非阻塞 I/O 的解决方法

```
1 int fd = open("file", O_RDONLY | O_NONBLOCK);
2 while ((n = read(fd, buf, sz)) < 0 && errno == EAGAIN) {
3     // 没数据：注册一个 fd 监听到 epoll，然后 yield
4     epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &ev);
5     coroutine_yield();
6     // 下次被唤醒：epoll_wait 检测到 fd 可读
7 }
```

Listing 3: 非阻塞 read + 协程 yield

Linux 异步 I/O 的两条主线

epoll (2002)：基于 fd 的事件通知；适合所有走 fd 抽象的对象（socket、pipe、eventfd、signalfd、timerfd），不适合真正的文件 I/O。**io_uring** (2019)：内核态环形缓冲区，提交 / 完成两个队列；是真正解决磁盘 I/O 异步的方案，今天 Rust 的 Tokio、Java 的 Netty 都用它。站在操作系统设计者视角，所有原本阻塞的 syscall 都应该有非阻塞版本——这是 Linux 一直在做的事情。

2.4 本章小结

协程 / Goroutine = 用户态栈 + 编译器协作调度。启动代价从微秒降到纳秒，单进程支持百万级并发。但用户态调度无法从内核手里抢回控制权——所以协程世界里所有阻塞 I/O 都必须改成非阻塞 + 事件驱动。

⁴视频画面时间区间：00:41:00–00:41:15。

3 路径二：JavaScript / Promise / async-await

3.1 为什么浏览器选了”完全单线程”？

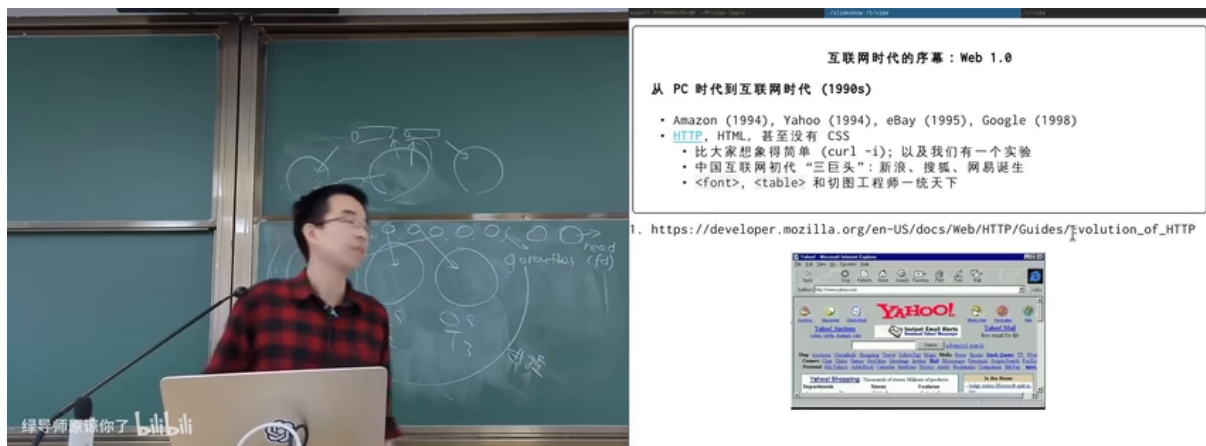


图 5: 互联网时代的序幕：Web 1.0 (1990s) ——Amazon (1994)、Yahoo!、eBay (1995)、Google (1998)。当时 HTTP / HTML ”简单到不需要 CSS”，所有页面都是后端 render 好后 curl 一次拿全；、<table> 和 弹窗是前端工程师的一统天下⁵

Web 1.0 → Web 2.0 的关键变化：**页面变得动态、交互、AJAX**——前端开始做真正的并发编程。但 JavaScript 的设计者做了一个非常激进的决定：

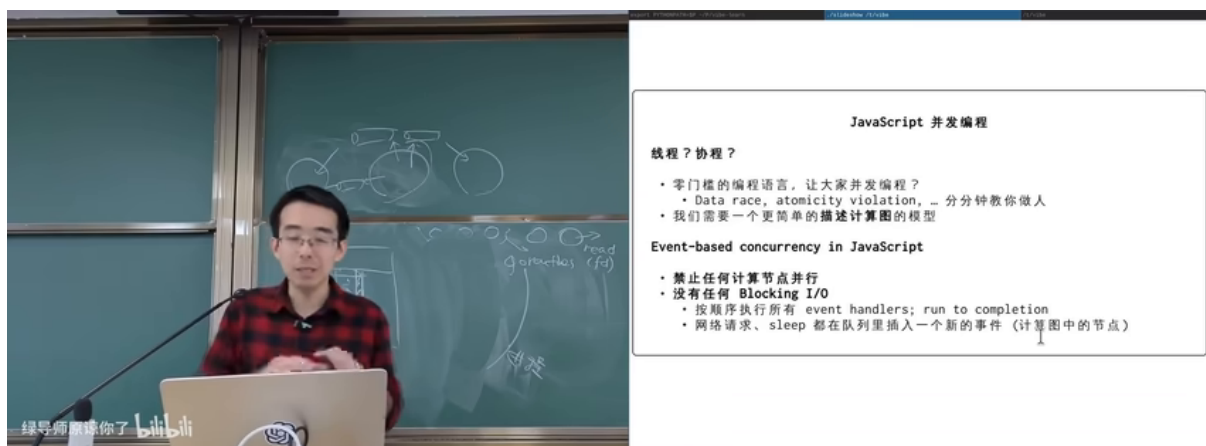


图 6: JavaScript 的并发选择：**完全单线程 + 事件驱动**。”线程？协程？零门槛的编程语言，让大家并发编程？Data race, atomicity violation...分分钟教你做人。”所以 JS 选了一个根本性更简单的模型——**Event-based concurrency**：禁止任何节点并行；没有任何 Blocking I/O；按顺序执行所有 event handlers, run to completion；网络请求 / sleep 都在队列里插入一个新事件（计算图中的节点）⁶

⁵视频画面时间区间：00:56:30–00:56:45。

⁶视频画面时间区间:01:04:30–01:04:45。

Event Loop 的关键不变式

在任何时刻，JS 的解释器都在执行恰好一个 event handler——直到它返回。之后才从事件队列里 dequeue 下一个事件。所以：

- 永远没有 race condition——只有单线程；
- 永远不能 sleep(5) 阻塞——会冻结整个浏览器；
- 所有“等待”必须改写成“发起请求 + 注册回调”。

代价：所有 I/O 必须是异步的；好处：零并发 bug，对前端工程师极其友好。

3.2 Callback Hell → Promise → async/await

阶段 1: Callback（极易写出“金字塔代码”）

```
1 fetchUser(id, function(user) {
2   fetchPosts(user.id, function(posts) {
3     fetchComments(posts[0].id, function(comments) {
4       // ...再嵌套几层就成 "callback hell"
5     });
6   });
7 });
```

Listing 4: Callback 嵌套

阶段 2: Promise（链式调用，但还是有点丑）

```
1 fetchUser(id)
2   .then(user => fetchPosts(user.id))
3   .then(posts => fetchComments(posts[0].id))
4   .then(comments => render(comments))
5   .catch(err => showError(err));
```

Listing 5: Promise 链

阶段 3: async / await（看起来就是同步代码!）

```
1 async function loadComments(id) {
2   try {
3     const user = await fetchUser(id);
4     const posts = await fetchPosts(user.id);
5     const comments = await fetchComments(posts[0].id);
6     return comments;
7   } catch (err) {
8     showError(err);
9   }
10 }
```

Listing 6: async/await 把异步写成同步样子

编译器对 `async/await` 做了什么？

编译器把每个 `await` 当作**协程**的 `yield point`：

1. 把 `async function` 编译成一个**状态机** (finite-state machine)；
2. 每个 `await` 是一个状态切换；
3. 等待的 `Promise` resolve 后，状态机回到下一个状态继续执行；
4. 局部变量”挂”到堆上（不能放栈上，因为函数会”返回”再”恢复”）。

所以你写的 `async/await` 在底层**和协程是一回事**——只是语言层面给你看起来像同步代码的语法糖。Python 的 `async def` / Rust 的 `async fn` / C# 的 `async/await` 用的都是同一套机制。

3.3 从计算图视角统一所有这些机制

两条路径其实是一回事

- **协程**：你写多线程代码，运行时把它**切成**事件驱动的状态机；
- **`async/await`**：你直接写状态机的 `yield point`，编译器**编排成**事件驱动的执行；
- **`Promise / Future`**：把**节点之间的依赖关系**显式表达出来——这就是计算图的边。

共同点：把”等待”从阻塞内核线程改成”等事件 + 让出 CPU”，让 1 个 OS 线程承担 100 万个逻辑任务。

3.4 本章小结

JavaScript 的极简之处在于”放弃并行换取零并发 *bug*”。`Promise / async-await` 是这个模型的语法糖，但它的语义到 Python、Rust、C# 都**完全可移植**——所有现代异步编程都站在同一个 `event loop + state machine` 的内核上。

4 总结与延伸

4.1 讲者核心观点

1. **OS 线程是太贵的并发单元**——上百万个并行计算节点根本用不起。
2. **两条工程路径**：协程让线程变轻，或直接重新设计执行模型为异步。
3. **协程的本质 = 用户态栈 + yield**——`setjmp/longjmp`、`makecontext`、`Boost.Context` 都是同一思路的不同实现。
4. **Goroutine M:N 调度**是 Go 把协程”产品化”的工程极致——单进程几百万 goroutine 是常态。
5. **JavaScript event loop** 选了相反的极端——完全单线程换取零并发 bug；`Promise / async-await` 是这个模型的现代语法糖。
6. **异步 I/O 是协程的命脉**——所有阻塞 syscall 必须换成 `O_NONBLOCK + epoll / io_uring`。
7. **两条路径在底层殊途同归**——都是把同一个”事件驱动的状态机”换不同的程序员接口。

4.2 自测三问

看完本讲，能答上这三题吗？

1. 用 Python generator 实现一个最小协程库：包含 `spawn`, `yield`, `join` 三个 API；解释为什么它能在单线程里跑 100 万个”thread”。
2. Go 的 goroutine 启动开销大约多少纳秒？为什么这么便宜？`GOMAXPROCS` 设大有什么后果？
3. 用 JavaScript / Python 的 `async/await` 写一个”先下载 3 个 URL，再并行处理 3 个结果，最后汇总”的流程；用 `Promise.all + await` 表达计算图。

4.3 与课程其他部分的 synthesis

- 与第 13-17 讲 (**多线程 / 互斥 / 信号量 / 条件变量**)：这些讲解决”OS 线程之间怎么协同”。本讲告诉你：当 OS 线程不够用时，你可以在一个 **OS 线程里搭建自己的”线程系统”**。
- 与第 18 讲 (**并行算法 / Scale Up**)：18 讲教你画计算图；本讲告诉你计算图节点的”低开销”实现方式。
- 与即将到来的第 20-21 讲：GPU 是另一种”轻量并发单元”的极致（每个 thread block 是协作式 warp）；Token 的旅程会让你看到大模型 serving 后端用 `io_uring + 协程批处理请求`。

4.4 拓展阅读

- Conway, *Design of a Separable Transition-Diagram Compiler*, CACM 1963 ——协程概念的源头。
- Dunkels et al., *Protothreads: Simplifying Event-Driven Programming of Memory-Constrained Embedded Systems*, SenSys 2006.

- Go runtime scheduler 文档: <https://go.dev/src/runtime/proc.go> (看 `schedule()` 函数)
- `man 7 epoll`、`man 2 io_uring_setup`、`man 2 eventfd`.
- MDN *Concurrency model and Event Loop*: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/EventLoop>.
- Rust *async-book*: <https://rust-lang.github.io/async-book/> ——把”`async/await` 怎么编译成状态机”讲得最清楚的一份教材.
- `Boost.Context`、`makecontext/swapcontext`、Linux `ucontext` 系列 API.